SECURITY · VOCABULARY

The Trust Vocabulary — Every Term on One Page

Permission · privacy · license vs grant · receipt · record of consumption · fingerprint · vault · token vs bookmark · key pair · mandate — one breath each, with who acts on it and the confusions that cost meetings.

For: anyone who has to say these words in a room — stakeholders, reviewers, IT, and the AI agents that parse this page. One definition per term, who acts on it, and the confusions that muddy meetings.

The whole system in one sentence: *privacy* consents → a *permission* or a *license* authorizes → a *grant* opens the *vault* door → the *record of consumption* writes who and from what source → the *receipt* proves it later, offline → the *fingerprint* stands in for anyone the system doesn't know.

Permission

What a subject **may do** in the platform: capabilities derived from roles (`crm:evidence:read`, `design:brand:theme`), granted by invitation, checked on every call, gone when the role changes. Nothing purchased, nothing durable. **Acts on it:** Owner/Admin grants · team members and agents hold · CISO governs the ruleset.

Privacy

What may be done **with a person's data**: consent categories, opt-outs, data rights — enforced fail-closed *in the data path*, not in the banner. The only object on this page whose owner sits outside the company. **Acts on it:** the customer owns · the DPO governs · every pipeline obeys.

License

The documented **right**: a signed, durable record of terms — who, what scope, what duration. The deed. It survives expiry, revocation, and key rotation,

because the record — not the file — is the product. **Acts on it:** the buyer owns · the platform issues · the auditor reads.

Grant

The operational **state of an open door**: access conferred on one subject for one resource, right now — revocable per subject in real time, checked at serve, every use ledgered. Time-boxed grants (`not_after`) are how "a month of access" is built. **Acts on it:** the platform enforces · Owner/Release confers and revokes · the subject exercises.

> **License vs Grant vs Permission — the scene:** the license is the *deed* (you bought a month of this asset); the grant is the *unlocked door* (the vault will serve you, this asset, until the 30th); the permission is the *job function* (you may pull evidence CSVs because you hold the audit role). One license can mint many grants; revoking a grant never rewrites the deed; permissions exist with no license at all.

Record of Consumption

What **actually happened**: the append-only ledger of serves, plays, scans — who, what, when, and from what source (channel stamp, origin, coarse geography), each row sealing the one before it. Royalty statement, compliance record, and marketing signal in one object. **Acts on it:** Data & Audit reads · Revenue BA reads for attribution · a regulator can subpoena it.

Receipt

The portable **proof** handed to the counterparty: a signed token (or its QR) that anyone verifies offline against the published public key — no account, no trust in the platform. It outlives the access it documents. **Acts on it:** the licensee carries · their IT verifies · nobody has to be believed.

Verify

The **act** performed on a receipt — by anyone, any number of times, forever: fetch the published public key the receipt names, test the signature, check the time window. No account, no call to the platform; verification works *against* the issuer, not through them. **The receipt is the claim; verify is the test the claim survives.** And each server-side verify is itself an event on the chain-of-custody register — checking the record extends the record. **Acts on it:** anyone — that is the point.

Fingerprint

The **stand-in for identity where none is shared** — two distinct senses, do not mix them:

1. **Actor fingerprint** — a salted, one-way hash marking a *distinct* verifier without storing who they are ("checked by 2 parties, 5 times").
2. **Key fingerprint** — the hash of a secret used to *verify a rotation happened* without the secret ever entering a log or a chat. **Acts on it:** the platform counts with the first · the Security Human ceremonies with the second.

Vault

The **custody plane**: encrypted storage whose only exit is the grant gate. Each asset sealed under its own key, versions immutable, every serve in the hash-chained ledger. The opposite of a file at a URL. **Acts on it:** Release deposits · grant-holders withdraw · Data & Audit reviews the chain.

Token

A sealed **credential you verify**: three base64url parts, signed by the platform's key, carrying its claims (subject, scope, expiry) inside itself. Bearer-ish while alive; dies on expiry or revocation; provable offline. **Acts on it:** the holder presents · any verifier checks against the public key set.

Session bookmark

A **pointer you look up**: the UUID a consent event mints, naming one visit's record. Contains nothing, grants nothing — like a receipt number, it says *which* record; the server decides *who* may read it. **Never call a bookmark a key. Acts on it:** the data plane joins on it · role-holders dereference it.

Short token

A **token built to die soon**: an access token that lives minutes or an hour, paired with a refresh token that buys the next one. Shopify's expiring offline tokens are the example: the access token lasts 1 hour, the refresh token is single-use and lasts 90 days, and every refresh returns a new pair and kills the old refresh token. A leaked short token is worth an hour, not a year. The cost is that exactly one process may refresh it, or two callers redeem the same refresh token and one of them is locked out. **Acts on it:** one refresher per store renews it · every caller reads the current one · nobody copies it into a `.env` file.

Long-run data stream

A **job that outlives its token**: a bulk export, a backfill, an event stream or a 15-minute middleware cycle that keeps running across hours. It can't hold one token for the whole run, so it asks the refresher for a fresh one mid-run, keeps a cursor so it can stop and resume, and writes idempotently so a retried batch doesn't double a record. Short token, long run: the credential expires on schedule while the data keeps moving. **Acts on it:** the job keeps the cursor · the refresher hands it tokens · the system of record dedupes every write.

TLS

The **sealed envelope for data in transit**: Transport Layer Security encrypts the connection and proves the server's identity with a certificate that chains to a root the client already trusts. It says *which server* you reached and that nobody read the wire; it says nothing about *who the caller is* or *what they may do* — that

is the token's and the entitlement's job. When a certificate authority's intermediates are revoked or its root is distrusted, every site chaining to it fails in browsers at once, so the issuing CA is a supply-chain dependency to review, not a detail. **Acts on it:** the CA issues · the server presents · every client verifies · the edge terminates and re-encrypts to the origin.

Key pair

The platform's **pen**: the private half ceremony-held and never leaving the server; the public half published for the world. It signs licenses, receipts, and mandates — one pair, thousands of signatures, rotated by named generations with no flag day. People and agents hold *signed documents*, not keys. **Acts on it:** the Security Human ceremonies · everyone else only ever touches the public half.

Mandate

A license's agent-shaped sibling: the signed document under which an **AI agent** may act — capped, scoped, time-boxed, revocable — so agentic checkout runs on authority that can be verified, not on a shared secret. **Acts on it:** the customer signs · the agent operates within it · any party verifies it offline.

Entitlement

The **account-shaped sum** of what a subject holds: features, capabilities, scopes — minted by purchase or role, carried by the identity spine, projected into tokens on demand. The bridge between "bought it" and "may do it." **Acts on it:** purchase mints · the platform projects · every gate consults.

The GitHub bridge (for anyone who knows git — that's everyone's IT)

You already trust this exact machinery every time you use GitHub; the platform just applies it to business records. Three mappings:

- **A commit SHA = the hash-chained ledger.** A git commit's hash includes its parent's hash — precisely the ledger's trick: each row seals the one before it, so history can be visibly broken but never quietly edited. Anyone can recompute the chain; no keys involved. A hash proves *what* — this exact content, untouched — but never *who*: a bare SHA has no author.
- **A signed commit's "Verified" badge = receipt + verify.** A signed commit is a hash sealed with a private key, and GitHub checks it against the published public key. That green badge is exactly what crm-sync.dev/verify does with a receipt: same math, same trust model, different subject matter.
- **The layering:** SHA-256 identifies (the firmware image, the record hash, each ledger row) · the signature attests (the receipt, the certificate, the mandate) · the chain orders it all in tamper-evident time. **Hash for *what*, signature for *who*, chain for *when*.**

One line for the room: *a SHA says "this is the thing, untouched"; a receipt says "this is the thing, untouched, and we sealed it — check for yourself."*

The confusions that cost meetings

- **A bookmark is not a key.** One is a pointer, one is a credential. Sharing a bookmark shares nothing; sharing a credential shares everything until revoked.

- **A license is not a grant.** Ending access (grant) does not rewrite the contract (license). Auditors read licenses; servers check grants.
- **A permission is not a license.** Roles are employment-shaped and mutable; licenses are purchase-shaped and durable.
- **A receipt is not the record.** The record is the platform's ledger; the receipt is the counterparty's proof. They corroborate each other precisely because they are held by different parties.
- **A fingerprint is not a signature.** A signature proves *authorship* of a document; a fingerprint proves *distinctness* (of an actor) or *possession* (of a key) without revealing anything else.
- **A hash is not a signature.** A hash (a git-SHA-style number) proves the content is untouched; a signature is a hash *sealed with the private key* and also proves who sealed it. Anyone can hash; only the keyholder can sign.
- **Possession is not authority.** The system's first principle. Everything above exists so that holding a thing — a link, a bookmark, even a token past its expiry — confers exactly and only what was signed.

Companion reading: Verify It Yourself — the IT Sheet · Rotate a Key in Three Steps · Firmware, SBOM & the Cyber Resilience Act — and the live proof at crm-sync.dev/verify.
